Portfolio 2026

형성희

Hyeong Seonghui

Software Engineer · React · Next.js · AI

사내 시스템부터 15개국에 출시한 앱까지, 화면 · 데이터 · AI를 직접 만드는 개발자

현재 Korea Digital 재직

comely1515@gmail.com · 010-3050-0299 · github.com/rapha155 · comelylabs.com

About

사용자의 자리에서 출발해, 화면부터 데이터까지 끝까지 책임집니다.

저는 화면을 만들기 전에 그 화면을 쓰는 사람의 하루를 먼저 봅니다. 엑셀을 옮기던 담당자, 탭 7개를 오가던 연구원, 클라우드를 쓸 수 없는 교실처럼, 문제는 늘 그분들이 일하는 흐름 안에 있었습니다.

기획부터 데이터 구조, 배포까지 직접 맡아 문제를 풀어 왔습니다. AI는 판단을 돕게 하고 계산과 검증은 코드가 책임지게 합니다. 무엇을 바꾸든, 이전과 같은 결과가 나온다는 것을 테스트로 확인한 뒤에야 넘깁니다.

약 2,000명
출시 앱 4종 누적 사용자

15개국
Noahlog 출시

22.9초 → 0.12초
주문·정산 관리자 조회 속도

1,768건
주문 누락 없이 이관

프론트엔드

React · Next.js로 화면을 만듭니다. 상태는 한곳에 모으고(zustand · 리듀서), 폼은 입력 규칙으로 검증하며(react-hook-form · zod), 공용 UI와 테마를 하나의 기준으로 관리합니다. 접근성 검사 위반 0.

→ PHRI 수직농장 · Science# · IT News CMS

백엔드 · 데이터

조회 API 없는 ERP 연동, 거래처 9,554건 · 주문 1,768건 이관, 22.9초 쿼리를 0.12초로 줄인 최적화를 직접 맡았습니다.

→ ERP 연동 주문·정산 관리자

AI 적용

AI가 앱을 안전하게 다루는 도구 계층과 프롬프트 정책, 법령 검색용 하이브리드 RAG를 설계했습니다. 개발에는 AI 코딩 에이전트를 지침과 테스트로 묶어 활용합니다.

→ Science# 로컬 AI · 공인중개사 AI 챗봇

제품 · 운영

Flutter 앱 4종의 기획부터 스토어 운영까지 전담했습니다. 사용자 의견을 업데이트에 반영합니다.

→ LingoStep · 한국사 · Noahlog · 말씀나무

React · Next.js · TypeScript · Tailwind · Vite · Node.js · FastAPI · PostgreSQL · AWS Lambda · CloudFront · Docker Compose · Firebase · Flutter · LangChain · Qdrant

Timeline 2023.09 ~ 현재

구분	기간	무엇을
Korea Digital AI 엔지니어 · 개발자 2026.07 ~ 현재	2026.10	**PHRI 수직농장** 연구원이 쓰는 스마트팜 운영 화면 UX · 팀(프론트 담당)
	2026.09 ~ 2026.10	**ERP 연동 주문·정산 관리자** 사내 주문·견적·정산 시스템 재구축 · 단독
	2026.07 ~ 2026.09	**Science# 로컬 AI** 로컬 LLM이 과학 실험 차트를 다루는 학습 도우미 · 팀(센서 통신 외 전체)
프로젝트	2025.12 ~ 현재	**Next.js 작업 4종** rapha-studio · IT News CMS(2026.04~) · 카드뉴스 미리보기 도구(2026.09~) · 한국사 앱 웹 버전
	2025.11 ~ 2026.03	**공인중개사 AI 챗봇** RAG 학습 → 하이브리드 RAG · FastAPI · Docker Compose
	2025.09 ~ 2026.08	**자격증 학습 플랫폼** SQLD · 한국사 · 모노레포 · PDF 파싱 자동화
	2025.03 ~ 현재	**한국사** 기획·개발 → 2025.05 출시 → 운영 중
	2025.01 ~ 현재	**SAYON → LingoStep IELTS** 기획·개발 → 2025.03 출시, 2026.10 IELTS 학습 앱으로 리뉴얼
	2025.01 ~ 현재	**말씀나무** 기획·개발 → 2025.03 출시 → 운영 중
	2024.06 ~ 현재	**Noahlog** Flutter 독학·기획 → 2024.09 15개국 출시 → 운영 중
	2024.02 ~ 2024.06	**운동기록 사이트(팀)** React · Spring 독학 후 팀 프로젝트
	2023.09 ~ 2024.02	**서비스 기획(팀)** 유튜브 프리미엄 UX 개선, 이력서 관리 서비스 기획(Figma)

Projects 6개



ERP 연동 주문·정산 관리자

주문·견적·정산 사내 시스템을 React + AWS로 재구축

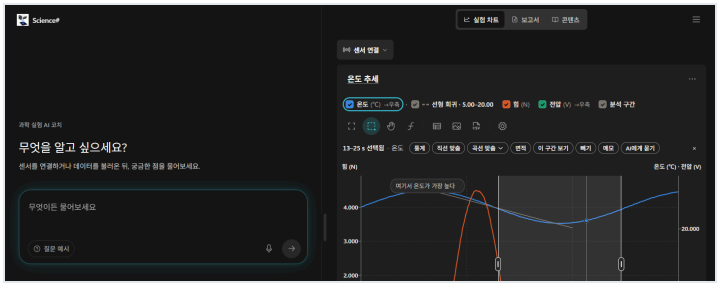
회사 · 단독 2026.09 ~ 10



PHRI 수직농장

연구원이 쓰는 스마트팜 운영 화면 UX

회사 · 팀(프론트 담당) 2026.10



Science# 로컬 AI

로컬 LLM이 과학 실험 차트를 다루는 학습 도우미

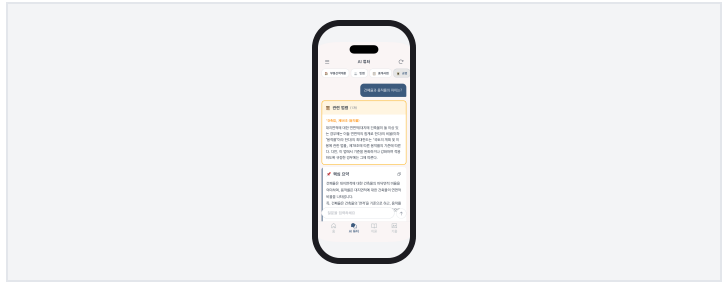
회사 · 팀(센서 통신 외 전체) 2026.07 ~ 09



IT News CMS

Next.js App Router로 만든 기사 운영 CMS

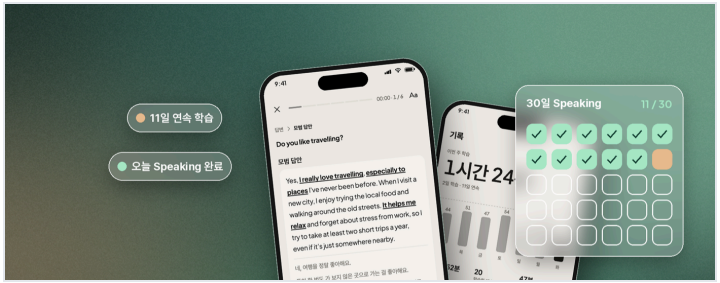
개인 2026.04 ~ 10



공인중개사 AI 챗봇

법령 조항을 정확히 찾는 하이브리드 RAG

개인 2025.11 ~ 2026.03



출시 앱 4종

LingoStep IELTS · 한국사 · Noahlog · 말씀나무

개인 · 운영 중 2024.09 ~

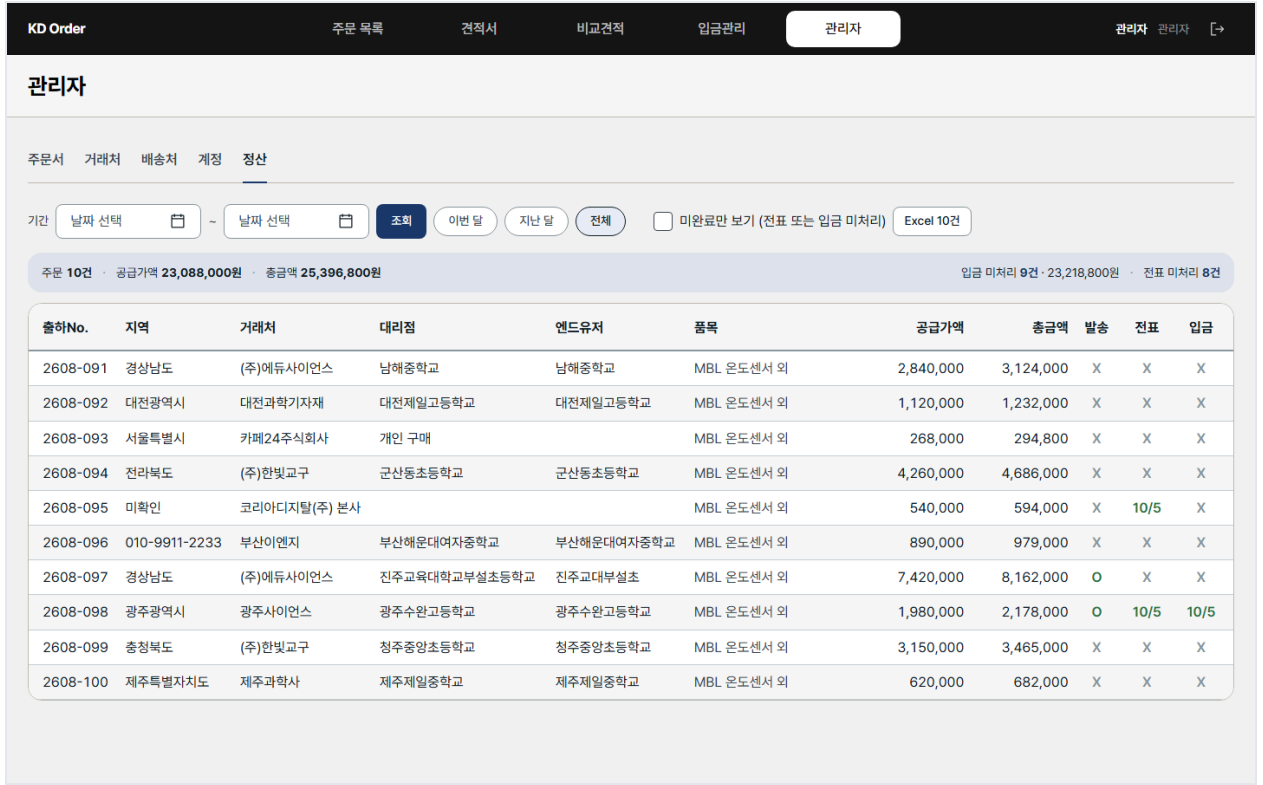
ERP 연동 주문·정산 관리자

Google Sheets와 Apps Script로 운영하던 사내 주문·견적·정산 업무를 React + AWS 웹 시스템으로 재구축했습니다.

기획 배경 주문 원천인 ERP(이카운트)에서 담당자가 매번 엑셀을 내려받아 옮겼고, 같은 업무 규칙이 화면마다 다른 답을 냈습니다.

핵심 문제 조회 API가 없는 ERP에서 데이터를 자동으로 가져오고, 흩어진 업무 규칙을 한곳으로 모으는 것

역할	프론트엔드 · 백엔드 · 배포 단독
기간	2026.09 ~ 10 · 6주
기술	React · TypeScript · Node.js Lambda · PostgreSQL · CloudFront



관리자 › 대리점 정산. 조회 22.9초 → 0.12초로 개선한 화면

엑셀 수작업을 자동 수집으로

API가 없는 ERP를 웹 통신 분석으로 연동

정산 조회 22.9초 → 0.12초

개선 전후 결과가 같음을 전수 비교로 확인

주문 1,768건 누락 없이 이관

적재 오류 68건 → 0건

ERP 연동 주문·정산 관리자

핵심 과제 3

API 없는 ERP에서 자동 수집

문제 주문 원천인 ERP(이카운트)에 데이터를 꺼내는 기능이 없어, 담당자가 매번 엑셀을 내려받아 올림

해결 웹 로그인 · 전송 형식을 분석해 조회 전용 클라이언트 구현. 기존 엑셀 적재의 미리보기 → 적재 경로에 연결, 비밀번호는 로그 마스킹

결과 **기간만 고르면 주문서·거래처가 자동으로 들어옴. 엑셀을 옮기던 수작업이 사라짐**

정산 조회 22.9초 → 0.12초

문제 대리점 정산 화면이 열리는 데 22.9초. 행마다 1만여 개 거래처에서 주소를 두 번씩 처음부터 찾고 있었음

해결 행마다 2회 호출되던 LIKE 검색 함수를 CTE로 1회로 줄이고, 주소 캐시 테이블 + 트리거로 갱신. 캐시 미스는 즉시 계산해 정확성 유지

결과 **0.12초로 단축. 바꾸기 전과 후의 결과를 모든 칸 비교해 하나도 다르지 않음을 확인**

흩어진 업무 규칙을 한곳으로

문제 발송상태처럼 같은 질문에 화면마다 다른 답. 규칙이 시트 수식과 스크립트에 흩어져 있음

해결 업무 규칙은 PostgreSQL 뷰 · 함수로 모으고 화면은 표시만. 목 데이터 ↔ 실서버는 DI 컨테이너 한 파일에서 교체, 1년치 주문은 전표번호를 키로 일괄 upsert

결과 **같은 규칙이 모든 화면에서 같은 답. 이관 오류 68건 → 0건, 견적서 금액 오류 0건**

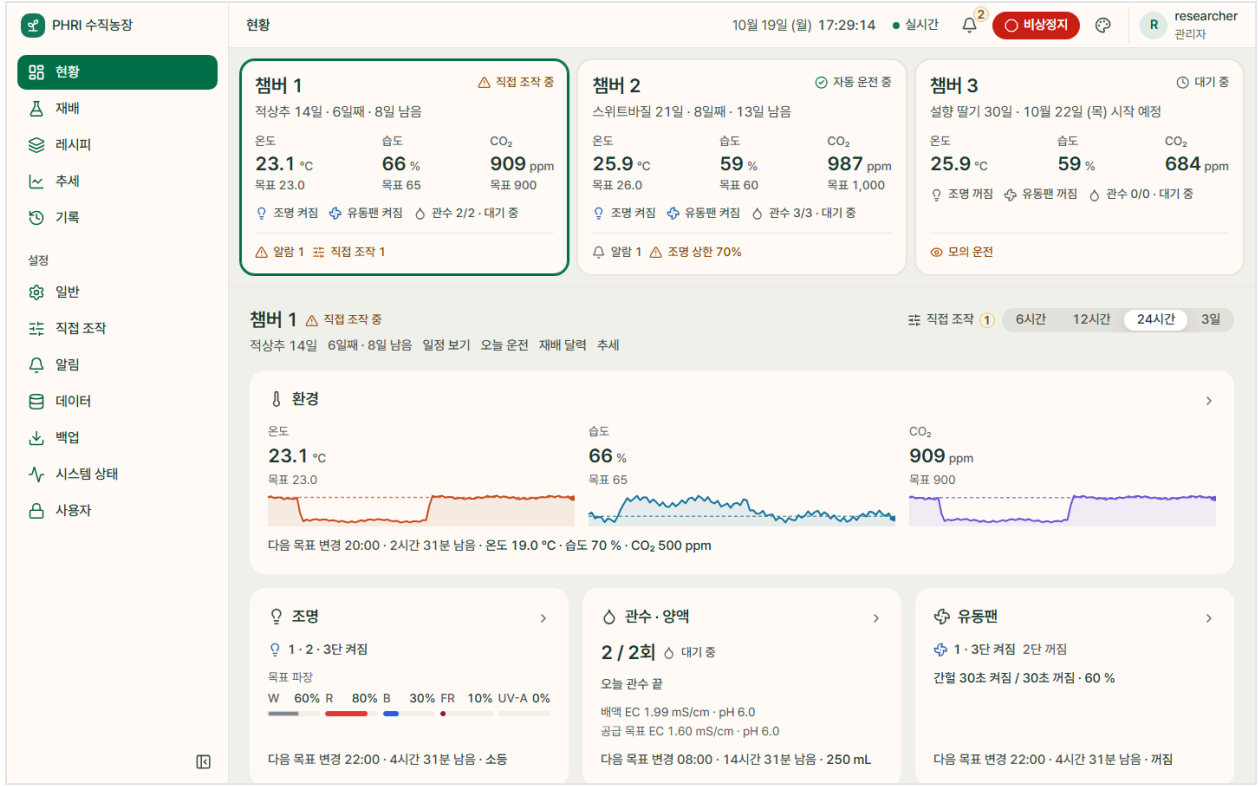
PHRI 수직농장

연구원이 수직농장 챔버 상태를 한 화면에서 확인하고 재배 일정과 알람을 다루는 운영 웹앱의 화면을 다시 설계했습니다.

기획 배경 세 번째 챔버가 스크롤 아래로 잘리고 상세 정보는 탭 7개로 나뉘어, 지금 전체가 정상인지 한눈에 볼 수 없었습니다.

핵심 문제 보는 화면과 조작 화면을 나누고, 정상은 조용히, 예외만 드러나도록 정보 구조를 다시 짜는 것

역할	프론트엔드 1인 담당 · 팀 프로젝트
기간	2026.10 · 10일
기술	React · TypeScript · zustand · react-hook-form · react-aria · orval



현황. 모든 챔버를 한 줄에, 예외만 표시. 탭 7개를 다이얼로그로 대체

챔버 상세 탭 7개 → 한 화면

노트북 화면 한 장에서 스크롤 없이 전체 확인

접근성 검사 위반 0

23개 화면 자동 검사, 다크 테마 글자 대비 포함

연구원 UI 제안 28건 반영

백엔드가 필요한 항목은 따로 정리해 전달

PHRI 수직농장

핵심 과제 3

서버가 바뀌어도 화면은 그대로

문제 새 서버는 데이터 형태가 기존과 달랐음. 화면을 하나하나 고치면 시간이 들고 실수가 생김

해결 orval로 API 클라이언트를 생성하고, DTO → 화면 모델 변환은 순수 함수 어댑터로 분리. 데이터 출처는 isStubApi() 한 곳에서 분기

결과 **겹치는 4개 화면을 화면 코드 수정 없이 새 서버에 연결. 개념이 다른 3개 화면은 이유를 기록하고 보류**

'정상처럼 보이는 오류' 제거

문제 불러오기에 실패해도 '0건'으로 보이고, 끊긴 연결의 지난 값이 현재 값처럼, 일부 선택이 전체 해제처럼 보임

해결 마지막 수신 시각으로 '서버 응답 없음' 경고와 흐림 처리, 알 수 없는 개수는 숨김. 공용 Checkbox에 indeterminate 상태 추가

결과 **연구원이 오래된 값을 현재 값으로 오해하지 않음. 공용 UI 수정 6건은 다른 앱에도 적용**

UX 문서를 먼저 쓰고 구현

문제 시안과 구현의 차이, 내부 용어가 섞인 화면 문구, 휴대폰에서 넘치는 표 5개

해결 UX 문서 15개(버튼 위계 · onTouched 검증 · 문구 규칙)를 먼저 쓰고 구현. 상태는 zustand, 폼은 react-hook-form + zod, 접근성은 react-aria

결과 **화면마다 다르던 버튼·문구·오류 표시가 하나의 규칙으로. 휴대폰에서도 표 넘침 없이 조작**

Science# 로컬 AI

학생이 센서로 실험을 측정하고, 차트를 분석하고, 로컬 LLM과 대화하며 보고서까지 쓰는 과학 수업용 웹앱입니다.

기획 배경 학교망과 개인정보 제약으로 실험 데이터를 외부 클라우드 LLM에 보낼 수 없었습니다.

핵심 문제 작은 로컬 모델이 계산을 틀리고 도구 호출을 망가뜨려도 측정 데이터와 화면이 손상되지 않게 하는 것

역할	센서 통신을 제외한 전체 개발 · 팀 프로젝트
기간	2026.07 ~ 09 · 7주
기술	React · TypeScript · Vite · LM Studio · Gemma 4 · Playwright



다크 테마. 왼쪽 AI 채팅, 오른쪽 실험 차트 탭과 구간 선택

AI는 요청만, 계산은 앱이

AI가 낸 숫자는 쓰지 않고 앱이 계산·검증

필요한 규칙만 담는 프롬프트

질문 종류별로 규칙·도구를 골라 작은 모델도 잘 따르게

모델이 실수해도 데이터는 안전

원본 보관 · 깨진 도구 호출 복구 · 허위 완료 답변 탐지

Science# 로컬 AI

핵심 과제 3

작은 모델에 맞춘 프롬프트 설계

문제 작은 로컬 모델은 규칙이 많을수록 덜 지키고, 요청마다 처음부터 계산해 응답이 느림

해결 요청을 의도(kind)별로 라우팅해 필요한 규칙 · 도구 · tool_choice만 전달. 고정 정책은 앞, 매 턴 바뀌는 차트 요약은 뒤에 두어 프리픽스 KV 캐시 재사용

결과 **규칙을 덜어 낸 만큼 모델이 더 잘 지키고, 앞부분을 재사용해 로컬 계산 부담을 줄임**

AI가 틀려도 데이터는 그대로

문제 AI가 직접 계산한 값은 검산하면 틀렸고, AI 편집이 원본 측정값을 덮어씀. 아무것도 바꾸지 않고 '바꿨습니다'라고 답하기도 함

해결 function calling 도구 11종으로 LLM은 요청만, 실행 · 검증은 앱이 담당. 편집 전 원본 보관, 쓰기 없이 '바꿨다'는 답변은 가드로 탐지

결과 **학생이 보는 수치는 모두 앱이 계산한 값. 원본은 언제든 되돌릴 수 있음**

차트 화면을 다시 설계

문제 도구줄이 그래프를 가리고 툴팁이 데이터를 덮음. 코드도 한 파일에 800줄 넘게 뒤엉켜 고치기 어려움

해결 Playwright 시각 회귀 기준 이미지 12장으로 화면을 고정한 뒤, 상태는 리듀서로, 계산은 domain 계층으로 분리. 도구줄 분리, 툴팁 대신 크로스헤어

결과 **화면을 그대로 둔 채 코드를 절반 이하로 정리하고, 데이터를 가리지 않는 차트로 개편**

IT News CMS

해외 IT 기사를 수집해 한국어 초안을 만들고, 검수해서 발행하는 편집자용 운영 도구를 Next.js App Router로 만들었습니다.

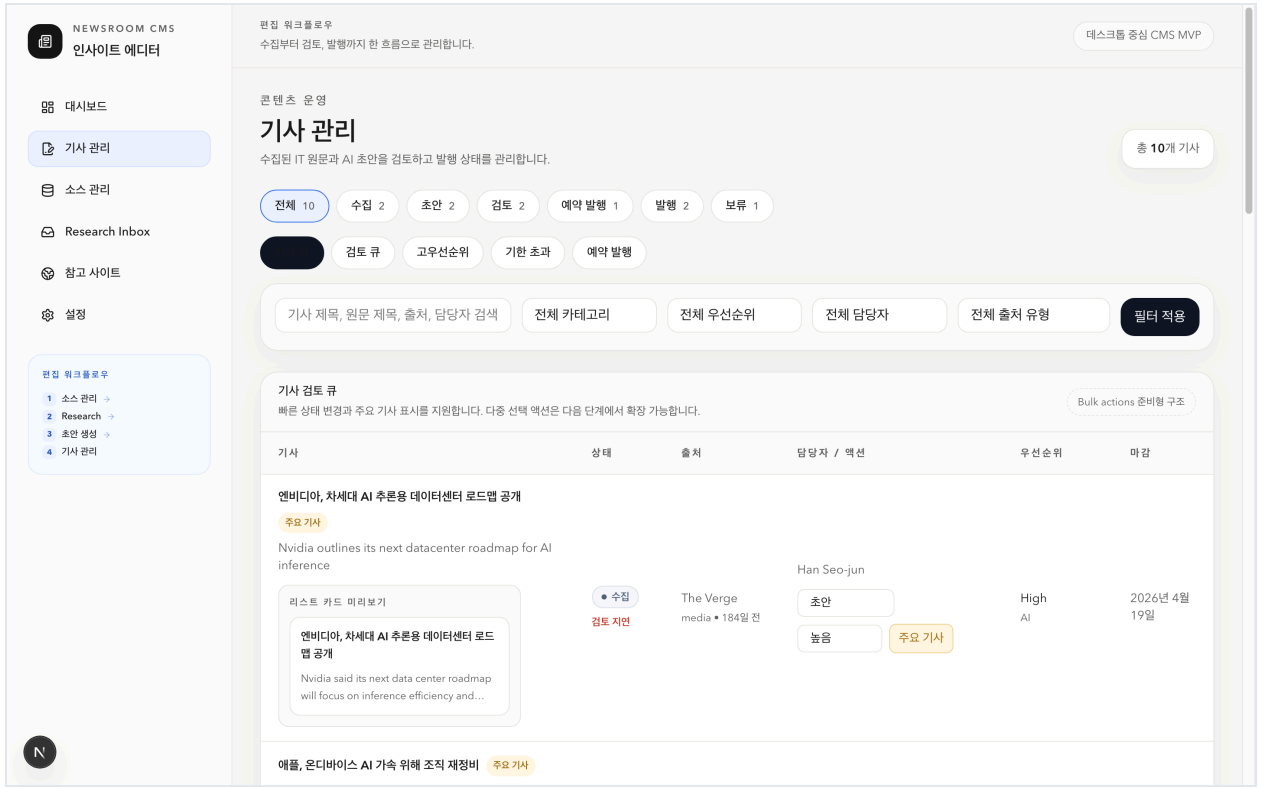
기획 배경 수집 · 번역 · 검수 · 발행을 거치는 기사가 지금 어느 단계에 있고 누가 맡았는지 한곳에서 볼 수 있어야 했습니다.

핵심 문제 단계가 많은 기사 상태와 복잡한 편집 화면을, 서버를 바꿔도 화면은 그대로인 프론트엔드 구조로 만드는 것

역할	기획 · 프론트엔드 단독
기간	2026.04 ~ 10
스택	Next.js App Router · React · TypeScript · Tailwind CSS

프론트엔드 구조

라우팅	라우트 그룹 (auth) · (protected), 동적 경로 [id], proxy로 접근 확인
컴포넌트	기능별 폴더: 기사 · 대시보드 · 리서치 · 소스
데이터	Route Handler + 저장소 계층, 샘플 → 실제 서버 교체 가능



기사 관리. 상태 탭과 운영 큐, 담당자·우선순위 필터

상태 6단계 기사 워크플로우

수집 → 초안 → 검토 → 예약 발행 → 발행/보류 전환 UI

3단 편집 화면

원문 근거 | AI 초안·편집본 | 검수 체크리스트, 미저장 변경 경고

용도별로 나눈 타입 모델

목록 · 상세 · 수정 요청 타입을 분리해 화면마다 필요한 데이터만

공인중개사 AI 챗봇

공인중개사 수험생이 법령 조항을 물으면 정확한 조문을 근거로 답하는 AI 튜터입니다.

기획 배경 법령 문제는 조항 번호까지 정확해야 하는데, 의미 검색만으로는 '제○조' 같은 질문에 엉뚱한 조문이 나왔습니다.

핵심 문제 조항 번호 같은 정확한 키워드와 뜻이 비슷한 질문을 함께 잘 찾고, 비슷한 질문에는 빠르게 답하는 것

역할	기획 · 백엔드 · 앱 단독
기간	2025.11 ~ 2026.03
스택	Python · FastAPI · LangChain · Qdrant · Elasticsearch · OpenAI · Flutter
환경	Docker Compose로 PostgreSQL · Qdrant · FastAPI 서버 구성





조항 번호 질문도 정확히 찾음

뜻으로 찾기와 단어로 찾기를 합쳐 순위를 정함

질문마다 하던 계산 3초 → 0.05초

문서 분석을 미리 해 두는 사전 인덱싱

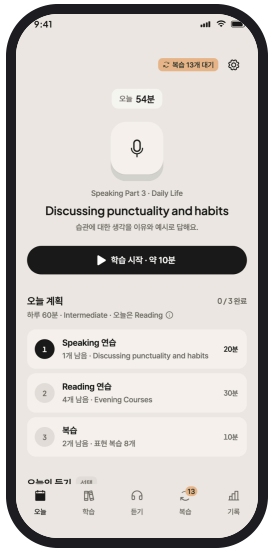
비슷한 질문은 바로 답함

저장해 둔 답을 다시 써 약 0.01초, AI 호출 없음

프로젝트 6 개인 · 출시 앱

스토어에 출시한 앱 4종

App Store · Google Play에 배포해 누적 사용자 약 2,000명이 쓴 앱입니다. 기획 · Figma 디자인 · Flutter 개발 · 배포 · 운영을 전담했습니다.



LingoStep IELTS

2025.03 ~ 운영 중 · 2026.10 리뉴얼

실력과 시간에 맞춘 오늘의 계획, 간격 복습, 화면을 꺼도 이어지는 반복 듣기

백그라운드 오디오 재생 · TTS 음원 파이프라인 · Firestore 동기화

App Store



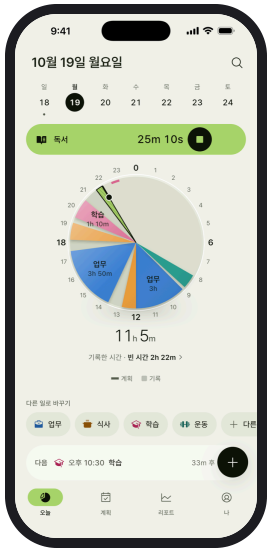
한국사

2025.05 ~ 운영 중

기출 PDF를 자동 구조화한 문제 풀이와 오답 재도전

기출 PDF 자동 파싱(PyMuPDF) · 이미지 캐싱 · 오답 기록

App Store · Google Play



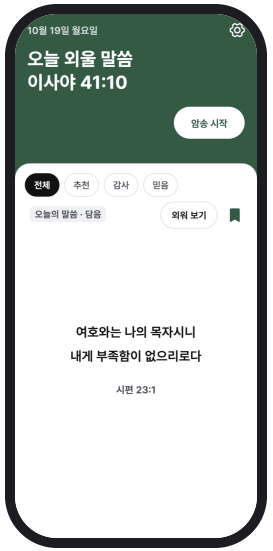
Noahlog

2024.09 ~ 운영 중

15개국에 출시한 시간 관리 플래너 · RTL 지원

오프라인 우선 동기화(SQLite · Firestore) · 시간 히트맵

App Store



말씀나무

2025.03 ~ 운영 중

매일 한 구절씩 외우고, 암송한 날이 쌓이는 성경 암송 앱. 홈 화면 위젯 지원

단계별 암송 세션 · 복습 일정 · 손님 → 소셜 계정 연결

App Store

How I Work

사용자의 말에서 시작합니다

쓰는 분들의 이야기를 먼저 듣고, 그 말을 기준으로 만듭니다. PHRI에서는 연구원 제안 28건을 하나씩 검토해 반영했고, 바로 할 수 없는 건 이유와 함께 전달했습니다.

다음 사람이 헤매지 않게 남깁니다

판단이 필요한 일은 근거와 선택지를 정리해 담당하는 분께 드리고, 정해진 결정은 바뀐 이유까지 기록합니다. 운영 중에 겪은 문제와 해결 과정도 문서로 남겨, 같은 문제를 두 번 겪지 않게 합니다.

결과가 같음을 확인한 뒤에 바꿉니다

성능을 개선하면 이전 결과와 모든 칸을 비교하고, 화면을 정리하면 기준 이미지와 맞춰 봅니다. 사용자가 모르고 지나칠 문제, 예를 들어 정상처럼 보이지만 틀린 값을 가장 먼저 찾습니다.

경계에 걸친 일은 먼저 정리해 건넵니다

팀 프로젝트에서는 맡은 범위를 분명히 하고, 다른 사람의 일과 맞닿는 부분은 먼저 정리해 전달합니다. PHRI에서는 백엔드가 필요한 요청을 따로 묶어 건넸고, 공용 UI를 고칠 때는 다른 앱에도 함께 반영되게 했습니다.

감사합니다.

사용자의 업무를 이해하고,
화면부터 AI까지 끝까지 책임지는 개발자로 함께하겠습니다.

Email	comely1515@gmail.com
Phone	010-3050-0299
GitHub	github.com/rapha155
Web	comelylabs.com

형성희 · Software Engineer